The poster, explained to someone new

Read the plain text aloud. The grey lines in brackets are notes to you: where to point, or when to pause. Every number in here is on the poster or in the paper.

If you only have 20 seconds

This poster is about teaching a computer to learn from its mistakes the way people do. When it fails a puzzle, I want it to work out *why* it failed before it changes anything, because different failures need different fixes, and the wrong fix can break things that used to work. I found a case where the program could have found the right answer but never looked at the puzzle the right way. I also found that the obvious fix made other puzzles worse. So the program has not solved new puzzles yet. What it has given me is a working way to diagnose a failure before fixing it.

Part 1. The story

Learning from failure is a very human thing

Think about how you learn. When you get something wrong, you do not just try the same thing again, harder. You ask: did I misread the question? Did I not know the method? Was there not enough information? Then you change *that* part. A child learning to stack blocks does this. A scientist whose experiment fails does this. A doctor does it too: diagnose first, then prescribe.

Most computer programs do not. When they fail, they either give up, or they do more of the same: search longer, try more guesses, use more computer power. My question is whether a program can do the human thing: notice *what kind* of failure it had, and change only the part that was actually missing.

The puzzles, and why they are a good test

[point to the small coloured grids, “demo 0” to “demo 3”]

The puzzles come from a test called ARC, created by the AI researcher François Chollet. Each puzzle is a small grid of coloured squares. You get a few examples, each one an input picture and the output picture it should turn into. You must work out the rule, and use it on a new picture.

Look at these four examples. Two yellow dots become a filled yellow square, and the dots mark its corners. So the rule is: draw the box whose corners are the dots, in the dots’ colour. Most people see that in a few seconds.

Chollet’s idea is that intelligence is not how many skills you already have. It is how quickly you can learn a *new* skill from very little. ARC is built to test exactly that: every puzzle has a new rule, and you usually get only two to five examples. You cannot pass it by memorizing. The version I use, ARC-AGI-2, is the harder second version. Chollet and Mike Knoop run the ARC Prize, which hosts this summit.

Where big neural networks struggle

Today’s most famous AI systems are big neural networks: deep learning, the technology behind chatbots and image generators. They are excellent at things that look like what they saw in training. ARC is designed to be hard for them, for a few simple reasons:

- They learn from huge amounts of data. Here each new rule comes with only a handful of examples.
- They are good at recognizing familiar patterns, and weaker at a rule they have never seen.
- They have no built-in exact check, so they can be confidently wrong.
- When they fail, it is hard to see why, so it is hard to fix the right thing. The usual fix is more data,

more training or more computing power.

To be fair: newer systems have improved a lot on the first ARC test, often by spending a great deal of computing on each puzzle, and many strong entries combine neural networks with search. Neural networks are not useless here. But on their own they do not tell you why they failed.

I took the opposite bet. My solver writes small, exact programs, and checks every program against every example. When it fails, it leaves a written record of what it tried. That record is something you can read and reason about. I also tried several neural pieces along the way: hints from a language model, learned ranking, a small recursive network and a search-guiding network. None of them helped enough in my setting, so I dropped them and wrote down why.

What others have done, and what is different here

Two well-known ideas come close to mine.

- **Counterexample-guided synthesis (CEGIS).** Guess a program, find an example where it breaks (a counterexample), use that to fix the guess, and repeat. It is "guess, check, fix".
- **DreamCoder** (Kevin Ellis and colleagues). It learns by writing programs and saving useful pieces as new building blocks, so its toolbox grows over time.

Both are important. My question is narrower: before fixing anything, decide *which layer* failed. Should the system search more, look at the picture differently, build a new tool, or admit that the examples are not enough? Then change only that layer, with checks that are fixed in advance and cannot be bent. That is a claim I can test and could lose. If my approach does not beat "guess, check, fix" and "grow a toolbox" under equal conditions, it is wrong. I have not run that comparison on real puzzles yet.

My journey: ten months of changing my mind

[optional: tell this part if the listener is interested; it is the human side of the work]

The work itself followed the same loop: try, fail, ask why, change the hypothesis.

- **December 2025.** A first prototype that used language-model hints. It was a prototype only.
- **April 2026.** I restarted with made-up puzzle worlds, where I know the answers. Checking every guess against the examples nearly eliminated false rules.
- **May to August 2026.** Real ARC puzzles. My solver went from 67 to 185 of the 1,000 practice puzzles, each one with a certificate.
- **The humbling part.** The same solver solved none of the 120 public test puzzles, and scored 0 on the online leaderboard. More hand-written tricks were clearly not enough. So I changed the goal: instead of adding skills by hand, could the system get what it was missing from its own failures?
- **September 2026.** The first try at that admitted nothing: 0 of 1,500 attempts. The reason was not bad luck. The new learner could only build what the old search could already reach.
- **Then a surprise.** The system's record of its own failures had been watching the wrong process, a simplified copy of the solver rather than the real one. Once repaired, the failures became informative.
- **Two opposite results.** Simply *describing* a failure did not help choose a fix (0.585 against 0.591, so no gain). Asking *how each possible fix changes the failure* did help (0.641 against 0.580). That told me failure is useful as something to experiment on, not just to describe.
- **October 2026.** On made-up puzzles the full loop worked, in 27 of 30. On real puzzles, it reached the solver for the first time, but did not survive the strict check. The poster is about why.

Each step changed what I believed. That is the point: reasoning from failure is how people make progress, and I want a program that can do it too.

Part 2. The poster, panel by panel

1. The question

["The question", top left]

My question is: how can a program tell that its way of solving is not good enough, and get a better one, without cheating on the test that judges it? A failure can have different causes, and each needs a different fix:

- it did not look hard enough (*search*);
- it looked at the picture the wrong way (*representation*);
- it does not have the tool it needs (*capability*);
- the examples are not enough to pin down the rule (*evidence*);
- a correct answer was blocked by the safety checks (*certification*).

2. The CORA loop

[the diagram, left column]

CORA is the name of my system. The solver tries the puzzle. If it fails, I look at the record of what it did, decide what kind of failure it was, and choose a fix. The fix is tested with checks written down in advance, and only then can it be used. Solid boxes are built and measured. Dashed boxes are ideas not built yet. One honest point: in this work I decided the kind of failure by hand. The automatic version is not built yet.

3. Four separate questions

["Four claims, four kinds of evidence"]

I keep four questions apart:

- Did the new tool really cause the success?
- Did the record of the failure matter?
- Is the new tool truly new?
- Does it help later, on other puzzles?

The first two have early evidence on practice puzzles only. The third is not established. The fourth is not tested in this work.

4. What I measured

[the first bar chart, 27 / 30 against 12 / 30]

On made-up puzzles, the improved system completed 27 of 30, against 12 of 30 before. That is about made-up puzzles only.

[the second bar chart, which shrinks from 60 to 0]

On 60 real practice puzzles:

- the solver failed 54;

- in 31 it left useful clues about why;
- in 4, my system built a new tool from those clues and plugged it in;
- in 1, the solver accepted the tool and used it.

But that success failed the strictest check, called leave-one-out. I hide one example, rebuild everything from the rest, and see whether the hidden example still comes out right. It is like testing a student: if they can solve a worked example you hid from them, they understood it; if not, they may have just memorized. It passed 0 of 14 of these checks. So no new real puzzle was solved, and my official score stays at 185 out of 1,000.

5. A counterexample to “just search harder”

[the big blue-and-orange grid, middle column]

Why did it fail? The program can look at a picture in seven different ways, called views. One view says “each separate blob is an object”. Another says “all squares of one colour are one object”. The program only has room to search four views, in a fixed order.

With all the examples, only a couple of views pass a basic test, the right one among them, and the program finds the right rule, marked G. When I hide examples and leave only two, more views pass the test by coincidence and take the four places. The right view gets pushed out, so the program never even looks at the right answer. Searching longer would not help, because the right view is not on the list being searched.

Two of the four places were wasted on views that look exactly the same to the program. It is like a library with four reading desks: if two desks are taken by copies of the same wrong book, the right book never gets read.

6. Test the fix before you trust it

[the grid with red boxes, top right]

The obvious fix is to force the program to look at one particular view first. Before trying that on real puzzles, I tried it on 24 made-up puzzles where I know the answers. Each row is one puzzle; red means it got worse. It helped none and hurt five. Four that were right went wrong, and one wrong answer got accepted. So I did not use it. A fix that rescues your favourite example can quietly break others.

7. The rules any fix must follow

[“What a valid repair must respect”]

Any fix must play fair. When I hide an example to test the program, the program may not peek at it, not even indirectly. One idea I had would have peeked indirectly, so I only use it to learn, never as proof. Removing the duplicate views plays fair, but it would change what the program looks at in about half of the situations I checked, so it must be tested carefully first. I have not run it. I also found that hiding an example can occasionally make a view fail the basic test, 7 times out of 724, so I call that an observation, not a rule.

8. Could this work outside puzzles?

[the panel marked BIOLOGICAL APPLICATION: UNTESTED]

The decision-making part might work in other fields: diagnose the failure, choose a fix, check the fix. One example is predicting how cells respond to a drug or a gene change. The puzzle-specific parts would not carry over. This is only an idea: I have no biology data and no biology results.

9. The message

Before a reasoning system changes itself, it should find out what kind of failure it had. And every fix should be tested fairly, including on cases where it could do harm.

Part 3. What has been done, and what has not

Done and measured

- A puzzle solver that writes exact programs and certifies 185 of 1,000 ARC practice puzzles.
- On made-up puzzles: the full learn-from-failure loop worked in 27 of 30 cases, against 12 of 30 before.
- On 60 real practice puzzles: tools built from the solver's own failures reached the solver in 4, and one was accepted and used.
- A precise explanation of why that one failed the strict check: the right view was cut, not missing.
- A safety test showing that the obvious fix harms other puzzles, so it was not used.

Not done, or not yet

- No new real ARC puzzle solved, and no score improvement. The score is still 185 of 1,000 on practice puzzles, and none of the 120 public test puzzles.
- No proof that the system invents truly new ideas.
- No automatic failure diagnosis. On this poster I did it by hand.
- No fair comparison yet against "guess, check, fix" or "grow a toolbox" on real puzzles.
- The duplicate-removal fix is designed but not run.
- A larger test on 120 new puzzles is frozen and has not been run.
- No biology results.

Part 4. If they ask

"Did it solve any new puzzles?" Not yet. No new real puzzles.

"Isn't this just guess, check, fix?" It includes that. What it adds is deciding what kind of failure happened before choosing a fix. Whether that extra step really helps is the next experiment.

"How is it different from DreamCoder?" DreamCoder grows a toolbox. I am asking when the toolbox is actually the problem, and when the problem is search, the view, or too few examples.

"Why not just use a big neural network?" They are powerful, but they need lots of data, they cannot check themselves exactly, and they do not tell you why they failed. I wanted failures I can read and fixes I can test.

"So was it a bug?" In this case, yes: a limit in how the solver chooses its views. Telling that apart from a missing ability is exactly what the diagnosis is for.

"What would Chollet say?" Probably: show me that it learns new skills efficiently, with every cost counted. That is the right test, and I have not passed it yet.

"What happens next?" Test the duplicate-removal fix fairly, with safety checks, before any new real test.

Words you may need

ARC	a public set of picture puzzles that tests learning new rules from a few examples.
example pair	one input picture and the output it should become.
rule	what turns the input into the output.
solver, engine	the program that looks for the rule.
program synthesis	finding a rule by writing a small program.
neural network, deep learning	AI that learns patterns from large amounts of data.
view	one way of cutting a picture into objects.
leave-one-out	hide one example and check that the rest still give it correctly.
counterexample	a case where a guessed rule gives the wrong answer.
CEGIS	counterexample-guided synthesis: guess, check, fix, repeat.
DreamCoder	a system that grows its own toolbox of program pieces.
made-up (synthetic)	puzzles I generate myself, with known answers.
false acceptance	a wrong answer that passed the checks.